

Software Engineering SoSe 2026

FitFridge

Abschlusspräsentation

Gruppe 08

Vu Thanh Tung Paeper 2761945
Siraam Sinnarajah 2646750

Prof. Dr. Larissa Putzar,
Sune Maute,
Fin Engelbart

HAW Hamburg

Agenda & Übersicht

1. Grundlagen

- 01** Motivation, Problem / Lösung
- 02** Tech-Stack
- 03** Architektur

2. Features

- 04** Kühlschrank Übersicht
- 05** Produkte hinzufügen
- 06** Mahlzeiten-Tracker

3. Prozess & Qualität

- 07** Tests
- 08** Arbeitsweise, Lessons Learned

Motivation, Problem / Lösung

Das Problem

MyFitnessPal, Yazio & Co. sind Rückblickstools:
Sie protokollieren, was du gegessen hast.

Was fehlt:

Welche Lebensmittel liegen überhaupt da?

Wieviel habe ich noch?

Erreiche ich damit meinen Tagesbedarf gezielt?

Bausteine der Lösung

1

Digitaler Kühlschrank

Immer im Blick: was du hast, wie viele Nährwerte noch übrig sind.
Suche über OFF-DB via Text oder Barcode.

2

Mahlzeiten-Tracker

Überblick des täglichen Konsums: Tagesbedarf anpassen, Produkte
suchen + Reste -> Kühlschrank oder direkt aus Kühlschrank in den Tracker

Tech-Stack auf einen Blick

Schicht	Technologie	Warum
Backend	Python 3.10+, Flask 3	Minimalistisch
Datenbank	SQLite	Eine Datei, kein Migrationstool nötig
Frontend	Jinja2-Templates + Vanilla-JS	Kein Build-Tool, kein Framework
Externe Daten	OpenFoodFacts API (urllib)	Freie Produktdatenbank
Entwicklung	Claude (Anthropic)	Debugging, Unterstützung

Languages



● Python 56.8% ● HTML 23.5% ● CSS 14.2% ● JavaScript 5.5%

Wichtig: Die Datenbank wird bei jedem Serverstart neu aufgebaut (schema.sql → DROP/CREATE) und per seed.py mit Demo-Daten gefüllt: Login demo/demo.

Schichtenmodell

Browser: Jinja2-Templates + Vanilla-JS

HTML-Forms



routes.py Frontend-Blueprint



*_service.py Fachlogik



*_repo.py reines SQL



db.py (sqlite3) → schema.sql + seed.py

Routen kümmern sich nur um HTTP, die Fachlogik liegt in den Services, SQL ausschließlich in den Repos.

Dadurch sind Services ohne Flask testbar und Änderungen bleiben auf eine Schicht begrenzt.

Kühlschrank-Übersicht

Live-Nährwerte je Posten

Jeder Kühlschrank-Eintrag zeigt Marke, Menge und berechnete Nährwerte aus 100g-Werten $\times$ aktueller Menge (calculate_total_nutrition).

- Einheiten: g · ml · kg · l · stk, Stück über grams_per_piece (z. B. „1 Ei = 60 g“)
- Menge direkt ändern: Nährwerte rechnen sofort neu; ungültige Eingabe = No-op
- Gesamtsummen über alle Posten als “Donut”
- Löschen nur eigener Einträge (user-gscoped)

Nährwerte werden nie gespeichert, immer gerechnet.

Mozarella

Devondale • 9300639600628 • 2026-07-02

Menge: 125.0 g

Gesamtnährwerte: 421.2 kcal • 33.9g Protein • 31.5g Fett • 0.0g Kohlenhydrate

125,0

Menge ändern ×

Nutzer Anmeldung / Registrierung nötig

Ohne Anmeldung ist keine Nutzung möglich.
Kühlschränke aller Nutzer sind getrennt.

Jede User-Query ist besitz-gefiltert $\rightarrow$ keine fremden Daten

Makro-Übersicht

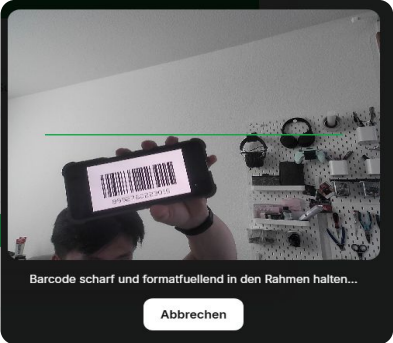


Produkte hinzufügen

1

Barcode, Scanner

BarcodeDetector API (nativ) → Fallback zxing-wasm
(Firefox/Windows/iOS)



Barcode oder Produktname

8992760223015

Barcode scannen

Die Treffer kommen von Open Food Facts. Findest du das Produkt nicht, kannst du es trotzdem mit "Trotzdem anlegen" trotzdem anlegen.

Oreo Chocolate Cream Cookies OFF **Hinzufügen**

Oreo 1a71a
Barcode: 8992760223015 · 493.0 kcal/100g

Trotzdem anlegen

Erst native API, dann Fallback.

2

Textsuche

search_products(): Textsuche bei Open Food Facts
_score_off_product(): Relevanz-Ranking (exakt /
enthält / Teilwort, Tiebreak Protein).

Barcode oder Produktname

Oreo Chocolate Creme Cookies

Suchen

Barcode scannen

Die Treffer kommen von Open Food Facts. Findest du das Produkt nicht, kannst du das Produkt mit "Trotzdem anlegen" trotzdem anlegen.

Oreo Chocolate Creme Cookies Biscuit 36.8gm OFF **Hinzufügen**

Oreo
Barcode: 7622201764555 · 478.260869565217 kcal/100g

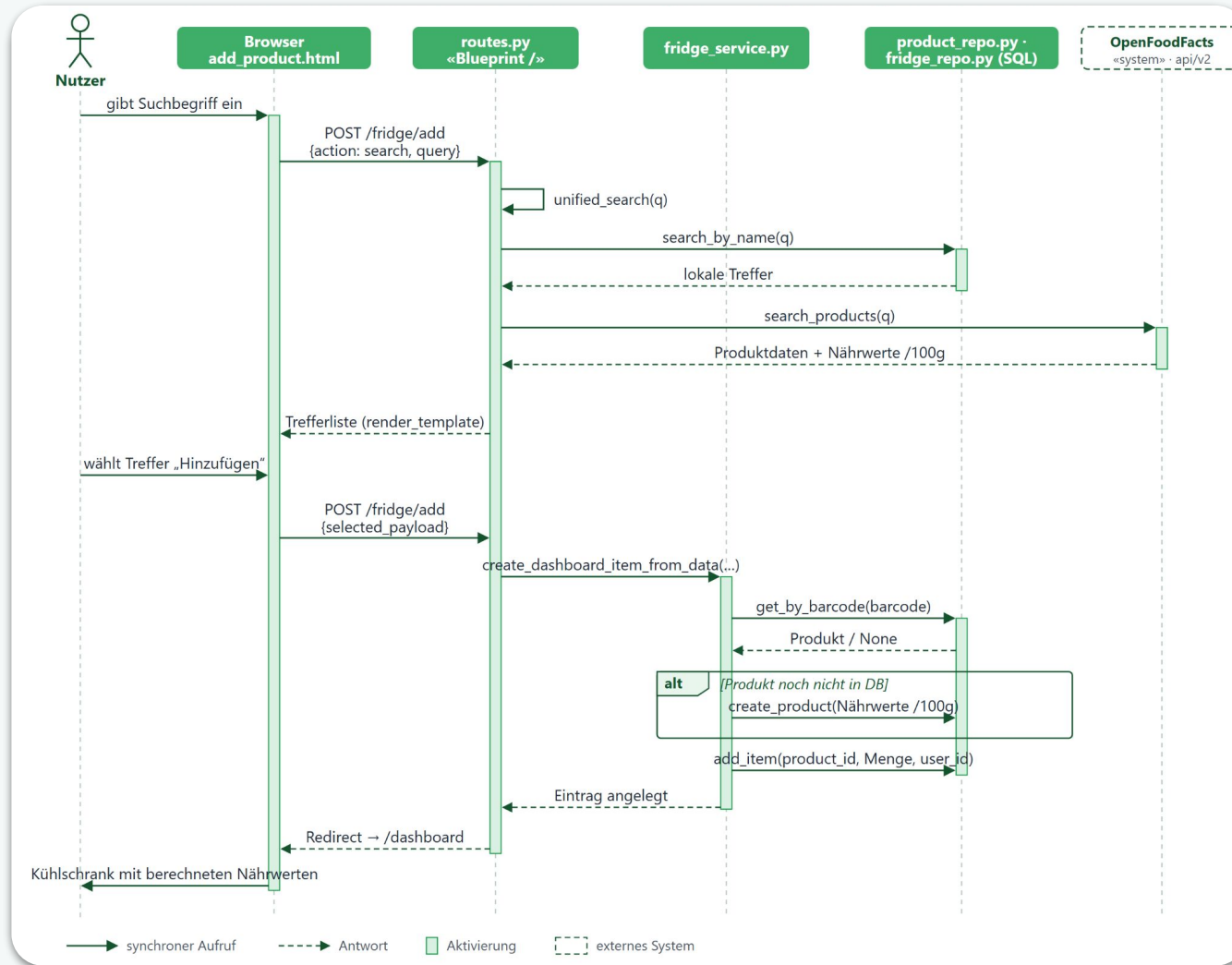
Oreo Dairy Milk OFF **Hinzufügen**

Cadbury
Barcode: 0061200014648 · 550.0 kcal/100g

Trotzdem anlegen

openfoodfacts_client.py

Produkte hinzufügen: Sequenzdiagramm



Zwei Quellen, erst lokale DB (search_by_name), dann OpenFoodFacts (search_products)

Produkte werden wiederverwendet: get_by_barcode() prüft vor dem Anlegen, create_product() nur, wenn das Produkt neu ist

Nur /100g-Werte gespeichert: Gesamtnährwerte werden nie persistiert, sondern bei jeder Anzeige live gerechnet

User-Scoping bis unten: add_item() bekommt die user_id, Eintrag gehört von Anfang an genau einem Nutzer

Externes System isoliert: OFF hängt nur am openfoodfacts_client; fällt es aus, gibt es leere OFF-Treffer statt eines Absturzes

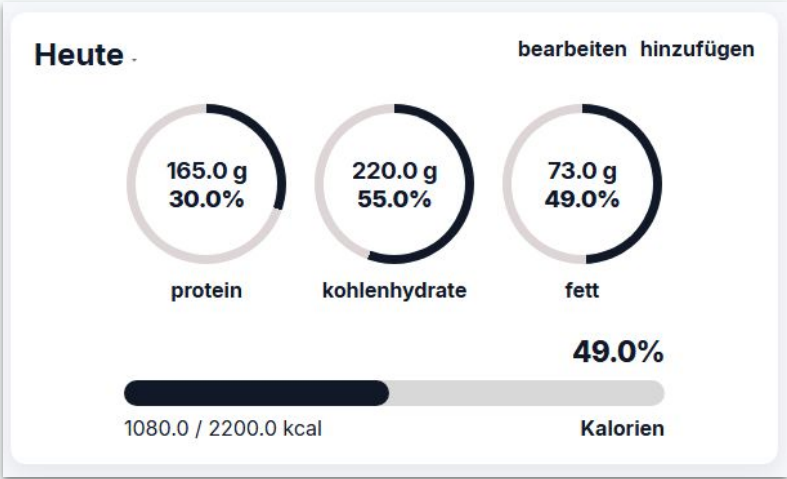
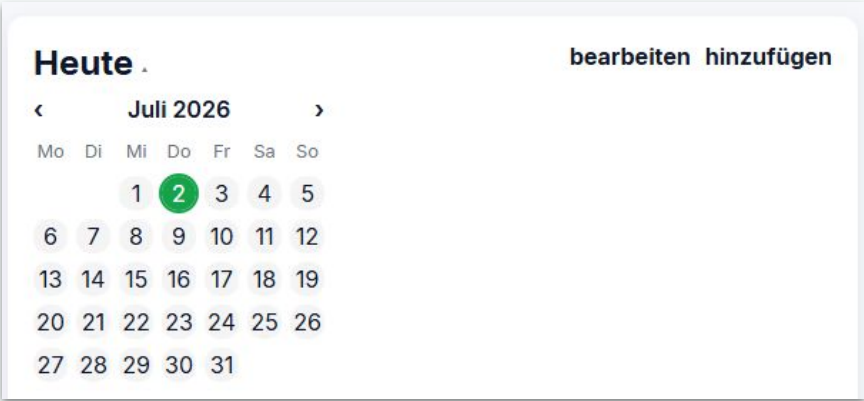
Mahlzeiten-Tracker

Kalender-Integration

calendar.monthrange liefert das Monatsraster, SQLite strftime markiert Tage mit Einträgen, für Langzeitbeobachtung.

Konfigurierbare Tagesziele

Setzen der Makroziele per Slider, immer 100% Gesamtmenge.



Mahlzeiten-Tracker

Warenkorb-Logik für das Hinzufügen

Beim Hinzufügen von Produkten: Neues Produkt suchen / scannen oder aus Kühlschrank.
Im Session-Warenkorb können diese gesammelt werden. `commit_meal_cart()` Fügt dann alle Produkte gemeinsam hinzu.

Reste in Kühlschrank

Beim Hinzufügen neuer Produkte können Teilmenge eingetragen werden, Rest in Kühlschrank

Hinzufügen

Aus Kühlschrank oder neues Produkt

Neues Produkt

Aus Kühlschrank

Produktname oder Barcode suchen

Mozarella

Suchen

Scannen

100

0

zur Mahlzeit

Mozarella

Devondale · 336.9980879541109 kcal/100g

menge übrig

75 75

zur Mahlzeit

Hinzufügen

Aus Kühlschrank oder neues Produkt

Mozarella

75.0 g · 75.0 g übrig → Kühlschrank

Mahlzeit erfassen

Neues Produkt

Aus Kühlschrank

zur Mahlzeit

Tomaten

FitFridge Demo

400.0 g

Menge

75

Zwiebel

FitFridge Demo

200.0 g

Menge

25

Paprika

FitFridge Demo

300.0 g

Menge

Hinzufügen

Aus Kühlschrank oder neues Produkt

Mozarella

75.0 g · 75.0 g übrig → Kühlschrank

Tomaten

75.0 g · aus Kühlschrank

Zwiebel

25.0 g · aus Kühlschrank

Mahlzeit erfassen

10

10 Tests

Tests

test_meal_tracker.py	6
test_nutrition_integration.py	2
test_api_db.py	2

OpenFoodFacts per monkeypatch ersetzt, alle Tests laufen offline.

Was die Tests absichern

① Mahlzeiten-Tracker: test_meal_tracker.py

- Tagesziele speichern, Prozente auf 100 % normalisiert:
`...settings_are_saved_and_percentages_can_be_normalized`
- Tagessummen & Restbedarf korrekt: `...daily_totals_and_remaining_values`
- Mahlzeit aus Kühlschrank zieht Bestand ab:
`...log_meal_from_fridge_item_deducts_stock`
- Mengenänderung skaliert Nährwerte:
`...update_meal_amount_scales_nutrition`
- Warenkorb-Reste landen im Kühlschrank:
`...cart_product_leftovers_go_to_fridge`
- Mahlzeiten gehören nur ihrem Nutzer:
``...meal_entries_belong_to_their_user``

② Fachlogik: test_nutrition_integration.py

- Nährwerte live aus $/100g \times \text{Menge}$; ungültige Menge $\rightarrow 0$

③ Externe Dienste: test_api_db.py

- Produkt/Fridge-Item-Roundtrip durch die DB
- OpenFoodFacts-Parsing (per monkeypatch, läuft offline)

Arbeitsweise, Lessons Learned

Kanban

Grobe Arbeitsteilung
Dann To-Do, In Work und Done
Wöchentliche Besprechungen für Progress, Probleme, Deadlines

Jede Person hat eigene Git Feature-Branches
Integration über Merge-Branches

Kommunikation: Discord, Whatsapp, wöchentliche Meetups

Lessons Learned

Feature-Branches pro Person + Merge-Branches: Integration wurde planbar, aber späte Merges waren teuer → früher und kleiner mergen

Abhängigkeit von externer API unterschätzt: ist OpenFoodFacts down, fällt die Produktsuche aus

Teamorganisation: Absprachen müssen verbindlicher sein

Klare Zuständigkeiten mit Deadlines pro Task

Fortschritt wöchentlich am Kanban-Board prüfen und bei wiederholt nicht eingehaltenen Zusagen früh eskalieren (Aufgaben umverteilen, Betreuer einbeziehen) statt Ausfälle stillschweigend zu kompensieren